

LIVRO TÉCNICO

Financial Control

Arquitetura, persistência e desenvolvimento do sistema
de controle financeiro pessoal

Sumário

Capítulo 5 — Anatomia do backend	1
..... 5.1 Estrutura e stack efetivamente usadas	1
..... 5.2 Configuração do Spring Boot	2
..... 5.3 Fluxo Controller → Service → Repository	2
..... 5.4 DTOs e fronteira HTTP	3
..... 5.5 Entities e JPA	4
..... 5.6 Ownership	4
..... 5.7 Transações, locks e concorrência	4
..... 5.8 Tratamento centralizado de erros	5
..... 5.9 Segurança do backend	6
..... 5.10 OpenAPI e Swagger	6
..... 5.11 Scheduler de faturas	6
..... 5.12 Testes do backend	6
..... 5.13 Como adicionar uma funcionalidade	7
..... 5.14 Anatomia resumida: transferência	7
..... 5.15 Pendências que não podem ser preenchidas por suposição	8
Capítulo 9 — PostgreSQL, modelo físico e Flyway	9
..... 9.1 PostgreSQL no Financial Control	9
..... 9.2 Como o Flyway evolui o schema	10
..... 9.3 Inventário das 27 tabelas	11
..... 9.4 Relacionamentos principais	12
..... 9.5 Constraints e índices	12
..... 9.6 Ownership em duas camadas	13
..... 9.7 Entities e enums	13
..... 9.8 Visões derivadas	14
..... 9.9 Testes do contrato físico	14
..... 9.10 Como criar uma nova migration	14
..... 9.11 Divergências encontradas na auditoria	15
Glossário	17
Índice remissivo	18

PARTE VI — BACKEND JAVA E SPRING

Capítulo 5 — Anatomia do backend

O backend do Financial Control é um monólito modular Java que expõe uma API REST sob `/api/v1`. Em agosto de 2026, a auditoria encontrou 270 arquivos Java de produção, 19 Controllers, 18 Services, 27 Repositories, 27 Entities e 112 DTOs. A execução da suíte terminou com 578 testes aprovados.

Ele é a autoridade das regras financeiras. Angular apresenta valores e melhora a experiência de preenchimento; saldos, remaining, limites, rateios e transições são decididos no backend.

5.1 Estrutura e stack efetivamente usadas

```
backend/
├── pom.xml
├── mvnw.cmd
├── src/
│   ├── main/
│   │   ├── java/br/com/financialcontrol/
│   │   │   ├── accounts/
│   │   │   ├── expenses/
│   │   │   ├── incomes/
│   │   │   ├── credit_cards/
│   │   │   ├── credit_card_invoices/
│   │   │   ├── ...demais domínios
│   │   │   ├── config/
│   │   │   └── security/
│   │   ├── resources/
│   │   │   ├── application.yml
│   │   │   └── db/migration/
│   └── test/
```

Tecnologia	Uso real
Java 25	linguagem e runtime
Spring Boot 4.1.0	configuração e composição da aplicação
Spring Web MVC	Controllers, JSON e HTTP
Spring Security	filtro stateless, JWT e Argon2
Spring Data JPA/Hibernate	persistência
Jakarta Validation	validação de requests
Flyway	criação e evolução do schema
Nimbus JOSE JWT	assinatura e leitura HS256
BouncyCastle	suporte ao Argon2id
springdoc-openapi	OpenAPI e Swagger UI
OpenPDF 3.0.5	relatórios PDF
JUnit/Testcontainers	testes com PostgreSQL real
Spotless	Google Java Format no <code>verify</code>

Não há H2, MapStruct, camada DAO adicional, UseCase por CRUD ou arquitetura hexagonal artificial.

5.2 Configuração do Spring Boot

O `application.yml` define:

```
spring:
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true

app:
  jwt:
    secret: ${JWT_SECRET}
    expiration-minutes: ${JWT_EXPIRATION_MINUTES:30}
```

`open-in-view: false` evita consultas lazy acidentais durante a serialização. `ddl-auto: validate` mantém o Flyway como única autoridade estrutural. `JWT_SECRET` não possui valor padrão: a aplicação falha cedo quando o segredo não está configurado.

`JacksonConfig` ativa `FAIL_ON_UNKNOWN_PROPERTIES`. Um campo JSON desconhecido em request não é ignorado silenciosamente; produz erro 400.

5.3 Fluxo Controller → Service → Repository

```
requisição HTTP
  ↓
Controller
  traduz HTTP, @Valid e principal autenticado
  ↓
Service
  regras, ownership, transação e orquestração
  ↓
Repository
  consultas e persistência JPA
  ↓
PostgreSQL
```

Nenhum Controller auditado acessa Repository diretamente.

Controller

O Controller conhece:

- URL e verbo;
- path variables e query parameters;
- request/response DTO;
- status HTTP;
- `AuthenticatedUser`;
- Service do módulo.

Exemplo representativo:

```

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public TransferResponse create(
    @AuthenticationPrincipal AuthenticatedUser user,
    @Valid @RequestBody CreateTransferRequest request) {
    return transferService.create(user, request);
}

```

Ele não calcula saldo e não decide se a transferência é válida.

Service

O Service concentra invariantes. Em uma transferência, ele valida:

1. origem diferente do destino;
2. data não futura;
3. ownership e atividade das contas;
4. tipo `BANK_ACCOUNT`;
5. saldo disponível;
6. ordem de locks;
7. persistência atômica.

Repository

Repositories são interfaces exigidas pelo Spring Data:

```

@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("""
    SELECT a FROM Account a
    WHERE a.id = :id AND a.userId = :userId
    """)
Optional<Account> findByIdAndUserIdForUpdate(UUID id, UUID userId);

```

Não existe DAO sobre o Repository.

5.4 DTOs e fronteira HTTP

Entities nunca são serializadas como contrato público. Requests e responses são records ou classes específicas do endpoint.

```

public record CreateTransferRequest(
    @NotNull UUID sourceAccountId,
    @NotNull UUID destinationAccountId,
    @NotNull
        @DecimalMin("0.01")
        @Digits(integer = 17, fraction = 2)
        BigDecimal amount,
    @NotNull LocalDate transferDate,
    String description) {}

```

Bean Validation verifica formato e limites estruturais. O Service verifica regras que dependem do estado persistido.

CONCEITO — DTO não é Entity reduzida DTO é contrato da fronteira. Ele protege o cliente contra alterações internas, impede mass assignment e permite que a API exponha derivados sem criar colunas.

Paginação usa o envelope:

```
{
  "items": [],
  "page": 0,
  "size": 20,
  "totalItems": 0,
  "totalPages": 0
}
```

O projeto não expõe diretamente `Page` do Spring Data.

5.5 Entities e JPA

Padrões observados:

- `@Table` alinhado ao nome plural da migration;
- ID UUID v7 atribuído pela aplicação;
- `user_id` em dados financeiros;
- `BigDecimal` com precisão 19 e escala 2;
- enum como `EnumType.STRING`;
- `Instant` para instantes e `LocalDate` para calendário financeiro.

Valores derivados não são propriedades persistidas. `AccountService` calcula saldo; `InstallmentBalanceService` calcula obrigação e remaining; Services de cartão calculam limites.

5.6 Ownership

O usuário vem do JWT:

```
@AuthenticationPrincipal AuthenticatedUser authenticatedUser
```

O Service consulta sempre pelo par:

```
repository.findByIdAndUserId(id, authenticatedUser.userId())
```

O body não escolhe o proprietário. Recurso alheio retorna 404, evitando revelar sua existência. O banco reforça o isolamento com FKs compostas.

5.7 Transações, locks e concorrência

Services de escrita usam `@Transactional`; leituras usam `@Transactional(readOnly = true)` quando apropriado.

Operações críticas usam `PESSIMISTIC_WRITE`. Transferências travam as duas contas em ordem determinística de UUID:

```
if (sourceId.compareTo(destinationId) < 0) {
    first = lock(sourceId);
    second = lock(destinationId);
} else {
    first = lock(destinationId);
    second = lock(sourceId);
}
```

Essa ordem reduz deadlocks. Testes de concorrência cobrem transferências, pagamentos de fatura e metas.

5.8 Tratamento centralizado de erros

O envelope real é:

```
public record ApiError(
    Instant timestamp,
    int status,
    String code,
    String message,
    String path,
    Map<String, String> fields) {}
```

Exceção	HTTP	Código
validação	400	VALIDATION_ERROR
regra de negócio	400	BUSINESS_RULE_VIOLATION
não encontrado	404	NOT_FOUND
não autenticado	401	UNAUTHORIZED
conflito	409	CONFLICT
inesperado	500	INTERNAL_ERROR

O projeto não usa RFC 7807.

Divergência: subcódigos não existem no JSON

Constantes como `SETTLEMENT_NOT_ALLOWED` e `SURCHARGE_REQUIRES_REMAINING` identificam regras no código e na documentação, mas a resposta contém somente:

```
{
  "code": "BUSINESS_RULE_VIOLATION",
  "message": "Mensagem da regra em português."
}
```

Não há campo `rule` ou `subCode`. Clientes não devem supor que o nome da constante é transmitido.

5.9 Segurança do backend

```

POST /auth/login
↓
AuthService verifica Argon2id
↓
JwtService cria token HS256, sub = UUID
↓
Authorization: Bearer ...
↓
JwtAuthenticationFilter
↓
AuthenticatedUser no SecurityContext

```

`SecurityConfig` usa sessão stateless, desativa CSRF para a API Bearer e configura CORS. São públicos health, registro, login e documentação OpenAPI. Os demais endpoints `/api/v1/**` exigem autenticação.

O token expira em 30 minutos por padrão. Não há refresh token, logout de servidor, OAuth, MFA ou roles na V1.

Senhas usam `Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8()`. `AuthService` compara um hash dummy quando o login não encontra usuário, reduzindo diferenças de tempo que poderiam revelar e-mails cadastrados.

5.10 OpenAPI e Swagger

Controllers usam `@Tag`, `@Operation`, `@ApiResponses` e `@SecurityRequirement`. Em desenvolvimento:

- OpenAPI: `http://localhost:8080/v3/api-docs`;
- Swagger UI: `http://localhost:8080/swagger-ui.html`.

As anotações ajudam descoberta, mas `docs/25-api.md` continua sendo o contrato detalhado de negócio.

5.11 Scheduler de faturas

`InvoiceClosingScheduler` executa:

```

@Scheduled(cron = "0 5 * * *", zone = "America/Sao_Paulo")
public void closeDueInvoices() {
    creditCardInvoiceService.closeDueInvoices();
}

```

O fechamento é idempotente e automático. Não existe endpoint normal para o usuário fechar fatura manualmente.

5.12 Testes do backend

A suíte auditada executou 578 testes sem falhas.

Categoria	Exemplos
API	<code>ExpenseApiTest</code> , <code>Phase14ApiTest</code> , <code>ReportsApiTest</code>
domínio puro	<code>InvoiceAllocationCalculatorTest</code>
schema	<code>SchemaContractTest</code>
ownership	<code>OwnershipAndPersistenceTest</code>
segurança	<code>JwtServiceTest</code> , <code>Argon2PasswordEncoderTest</code>
concorrência	<code>TransferConcurrencyTest</code>

Testcontainers sobre `postgres:18-alpine`. Isso testa SQL, migrations e constraints do banco oficial, evitando a falsa compatibilidade que um banco em memória poderia introduzir.

Lacunas pontuais de cobertura

- refund ACCOUNT/NONE com `settlement` é bloqueado no código, mas não possui teste dedicado encontrado;
- o bloqueio de edição de parcela já em fatura não tem teste dedicado;
- a tabela legada `credit_card_invoice_installments` ainda é exigida por teste de schema, embora não participe dos Services.

5.13 Como adicionar uma funcionalidade

1. Registrar requisito e regras nos documentos oficiais.
2. Parar se modelo, cálculo, status ou ownership não estiverem decididos.
3. Criar migration somente após a decisão.
4. Mapear Entity e Repository.
5. Criar DTOs de request e response.
6. Implementar Service transacional com ownership.
7. Criar Controller fino.
8. Documentar OpenAPI e contrato em `docs/25`.
9. Testar regra, persistência, HTTP, segurança e concorrência quando aplicável.
10. Executar `mvnw.cmd test` e `mvnw.cmd verify`.

NÃO FAÇA Não crie UseCase para cada verbo CRUD, interface e `Impl` para toda classe, mapper automático sem necessidade, nem aceite `userId` do cliente.

5.14 Anatomia resumida: transferência

HTTP

```
POST /api/v1/transfers
```

Fluxo

```
TransferController
  → TransferService
    → lock das contas em ordem UUID
    → valida BANK_ACCOUNT e saldo disponível
    → TransferRepository.save(ACTIVE)
    → trava saldo inicial das duas contas
```

O registro não grava saldos. `AccountService` inclui transferências ACTIVE ao calcular saldo. A reversão muda o status para REVERSED e preserva histórico.

Por que essa feature é didática

Ela atravessa todas as decisões centrais:

- DTO e `@Valid`;
- identidade vinda do JWT;
- ownership;
- Service transacional;
- duas linhas com lock;
- valor monetário `BigDecimal`;
- fato persistido e saldo derivado;
- ação de negócio por POST, sem DELETE;
- teste de efeito no saldo e teste de concorrência.

5.15 Pendências que não podem ser preenchidas por suposição

Tema	Estado
valores de <code>payments.type</code>	PENDENTE
edição cadastral de parcela já em fatura	PENDENTE/deferida
refresh token	não implementado por decisão de escopo
extrato unificado	planejado fora do escopo atual
remoção da tabela V13 superada	requer migration e decisão explícita

PARTE VIII — PERSISTÊNCIA E EVOLUÇÃO DO BANCO

Capítulo 9 — PostgreSQL, modelo físico e Flyway

Este capítulo descreve o estado físico encontrado no repositório em agosto de

1. O modelo é controlado por trinta migrations Flyway, de V1 a V30, e contém

27 tabelas. As visões de contas a pagar, contas a receber, projeções, dashboard e relatórios não possuem tabelas próprias.

ARQUITETURA — Uma única fonte para o schema O Flyway cria e evolui o banco. O Hibernate usa `spring.jpa.hibernate.ddl-auto=validate`: ele verifica o mapeamento, mas não cria nem corrige tabelas. Isso impede que o comportamento do banco varie conforme a inicialização da aplicação.

9.1 PostgreSQL no Financial Control

O banco oficial é PostgreSQL 18. No desenvolvimento, o Compose usa a imagem `postgres:18-alpine`. A escolha aparece simultaneamente em `docker-compose.yml`, no contrato de ambiente e nos testes Testcontainers.

As correspondências principais são:

Conceito	PostgreSQL	Java
Identificador	UUID	UUID
Dinheiro	NUMERIC(19,2)	BigDecimal
Instante absoluto	TIMESTAMPZ	Instant
Data financeira	DATE	LocalDate
Estado booleano	BOOLEAN, coluna <code>active</code>	boolean, propriedade <code>isActive</code>

UUID v7

Os identificadores normais são gerados na aplicação por `br.com.financialcontrol.UuidV7`. As entidades não usam `@GeneratedValue`, e as colunas não têm `DEFAULT` gerador. O projeto evita duas autoridades concorrentes para IDs.

A V30 usa `uuidv7()` do PostgreSQL somente durante o backfill histórico de `income_movements`. Isso não muda a política das operações normais.

Valores monetários

Dinheiro nunca usa `float` ou `double`. O banco fixa duas casas decimais e os Services normalizam valores com `RoundingMode.HALF_UP`.

Exemplo de parcelamento de R\$ 100,00 em três partes:

```
base = 100,00 / 3, arredondado para baixo = 33,33
primeira parcela = 100,00 - (33,33 × 2) = 33,34
demais parcelas = 33,33
soma = 100,00
```

O residual fica na primeira parcela. Há uma divergência importante para valores muito pequenos, documentada em 9.11.

Datas e timezone

`Instant` representa eventos absolutos e é persistido em UTC. `LocalDate` representa vencimentos, datas de compra, recebimento e pagamento. As regras de calendário usam `America/Sao_Paulo`; não dependem do timezone do navegador.

9.2 Como o Flyway evolui o schema

As migrations ficam em:

```
backend/src/main/resources/db/migration/
```

Elas são aplicadas em ordem numérica. Uma migration executada não é reescrita: mudanças posteriores usam uma nova versão.

Versão	Marco principal
V1	<code>users</code>
V2–V5	contas, categorias, cartões e metas
V6	receitas
V7	faturas no modelo inicial
V8–V10	despesas, parcelas e pagamentos
V11–V14	transferências, pagamentos de fatura, tabela posteriormente superada e contribuições
V15–V16	unicidade de categorias e responsável nullable em receitas
V17	status de payment e ajustes de parcela
V18–V25	cartões/faturas da Fase 9 e status <code>SETTLED_BY_AGREEMENT</code>
V26–V27	Agreements, settlements e alocações
V28	transferências, acertos e trava de saldo inicial
V29	vínculo de meta à conta e resgates
V30	movimentações de receita e backfill

Evolução de status sem apagar história

A V7 criou o primeiro CHECK de fatura. A V19 substituiu esse CHECK pelo conjunto `SCHEDULED`, `OPEN`, `CLOSED` e `PAID`; a V25 adicionou `SETTLED_BY_AGREEMENT`. A migration antiga permanece como registro da evolução, mas o contrato efetivo é o CHECK final.

Valores derivados não viram colunas

O schema não persiste:

- saldo atual, reservado ou disponível da conta;
- total pago ou remaining da parcela;
- totais de fatura;
- limite usado e disponível do cartão;
- valor acumulado e percentual de meta;
- resultados de payables, receivables, projections, dashboard ou reports.

Persistir esses valores criaria uma segunda fonte sujeita a dessincronização. Fatos — pagamentos, alocações, créditos e movimentos — são persistidos; totais são calculados.

9.3 Inventário das 27 tabelas

Cadastro e identidade

- `users`
- `accounts`
- `categories`
- `credit_cards`

Receitas

- `incomes`
- `income_movements`

Despesas

- `expenses`
- `expense_installments`
- `expense_installment_adjustments`
- `payments`

Cartões, faturas e créditos

- `credit_card_invoices`
- `credit_card_invoice_payments`
- `credit_card_invoice_payment_allocations`
- `credit_card_credits`
- `credit_card_credit_applications`
- `credit_card_invoice_adjustments`
- `credit_card_invoice_adjustment_allocations`
- `card_purchase_account_refunds`
- `credit_card_invoice_installments` — tabela legada, sem uso operacional atual

Agreements

- `credit_card_invoice_agreements`
- `credit_card_invoice_agreement_settlements`
- `credit_card_invoice_agreement_settlement_allocations`

Movimentação entre contas

- `transfers`
- `account_balance_adjustments`

Metas

- `financial_goals`
- `goal_contributions`
- `goal_redemptions`

9.4 Relacionamentos principais

```

users
├── accounts
├── categories
├── credit_cards — credit_card_invoices
├── incomes — income_movements — accounts
├── expenses — expense_installments
│               ├── payments — accounts
│               └── credit_card_invoices
├── transfers — accounts (origem e destino)
└── financial_goals — accounts
                        ├── goal_contributions
                        └── goal_redemptions

```

Uma despesa parcelada pertence à tabela `expenses`, mas cada parcela é um fato em `expense_installments`. Em compras no cartão, a parcela — e não a despesa — aponta para a fatura. Assim, uma compra em dez vezes pode atravessar dez ciclos.

9.5 Constraints e índices

O banco protege invariantes que não devem depender somente da aplicação.

Tipo	Exemplos
Chave primária	<code>id</code> UUID
Unicidade de ownership	<code>UNIQUE (id, user_id)</code>
CHECK de status	despesas, receitas, faturas, payments, movements
CHECK de alvo	coerência entre <code>payment_method</code> , conta e cartão
Unicidade de parcela	<code>(expense_id, installment_number)</code>
Uma fatura aberta	índice parcial único por cartão
Categoria sem duplicata lógica	índice por usuário, tipo e <code>lower(name)</code>

Índices de `user_id`, datas e FKs sustentam as consultas multiusuário. Índice não substitui regra de ownership: ele melhora busca; a FK composta preserva integridade.

9.6 Ownership em duas camadas

O Service sempre consulta com o `userId` extraído do principal autenticado:

```
accountRepository.findByIdAndUserId(accountId, authenticatedUser.userId())
```

O banco repete a proteção em relacionamentos:

```
FOREIGN KEY (expense_id, user_id)
  REFERENCES expenses (id, user_id)
```

Isso impede que uma falha futura no Service grave uma parcela do usuário A em uma despesa do usuário B.

SEGURANÇA — Por que retornar 404 Consultas por `(id, user_id)` tratam um recurso alheio como inexistente. Além de isolar dados, isso evita confirmar a existência de IDs pertencentes a outro usuário.

O JPA normalmente mapeia a coluna de relação simples. A FK composta permanece no Flyway. Essa separação mantém o mapeamento legível sem enfraquecer o modelo físico.

9.7 Entities e enums

Há 27 classes `@Entity`, uma para cada tabela. Não existem entidades JPA para payables, receivables, projections, dashboard e reports, porque esses recursos são visões calculadas.

Enums Java acompanham os CHECKs do banco:

- conta: `BANK_ACCOUNT`, `CASH`;
- receita: `EXPECTED`, `RECEIVED`, `CANCELLED`;
- despesa: `OPEN`, `PARTIALLY_PAID`, `PAID`, `CANCELLED`, `REFUNDED`;
- payment: `ACTIVE`, `REVERSED`;
- fatura: `SCHEDULED`, `OPEN`, `CLOSED`, `PAID`,

`SETTLED_BY_AGREEMENT`;

- meta: `ACTIVE`, `COMPLETED`, `CANCELLED`.

Duas exceções são intencionais:

1. `payments.type` continua `String`, sem enum ou CHECK, porque seus valores

oficiais permanecem pendentes;

1. `credit_card_invoice_installments.status` pertence ao modelo superado e não ganhou novo contrato.

9.8 Visões derivadas

Recurso	Implementação	Tabela própria
Payables	<code>PayablesService</code>	não
Receivables	<code>ReceivablesService</code>	não
Projections	<code>ProjectionService</code>	não
Dashboard	<code>DashboardService</code>	não
Reports	<code>ReportsService</code>	não

Uma linha de payable pode representar parcela ACCOUNT/NONE ou fatura com remaining positivo. Receivables deriva receitas e movimentações. Projeções combinam saldo atual e eventos futuros. Dashboard agrega serviços existentes. Reports calcula visões e, quando solicitado, produz PDF com OpenPDF.

9.9 Testes do contrato físico

Os testes usam PostgreSQL real em Testcontainers.

Teste	O que protege
<code>SchemaContractTest</code>	tabelas, CHECKs e ausência de colunas derivadas
<code>OwnershipAndPersistenceTest</code>	FKs compostas e isolamento
<code>Phase8PersistenceContractTest</code>	schema de parcelas, payments e adjustments
<code>V30IncomeMovementsBackfillTest</code>	migração e backfill de movimentos

O `SchemaContractTest` passou durante a auditoria. Isso comprova alinhamento com o contrato que o próprio teste expressa; não prova que toda regra documental esteja correta. A seção seguinte mostra por quê.

9.10 Como criar uma nova migration

1. Confirmar que a regra de negócio está decidida.
2. Escolher a próxima versão; nunca renomear ou editar uma migration aplicada.
3. Criar tabela ou `ALTER TABLE` com tipos, nulabilidade e constraints.
4. Incluir `user_id` e ownership composto quando o dado for financeiro.
5. Não persistir um valor definido como derivado.
6. Atualizar a Entity de forma compatível.
7. Criar teste de schema e teste de integração do comportamento.
8. Executar `.\mvnw.cmd test` e `.\mvnw.cmd verify`.

NÃO FAÇA Não use `ddl-auto=update`, não adicione `DEFAULT` gerador de UUID, não remova FK composta para simplificar JPA e não crie tabela para uma visão calculada.

9.11 Divergências encontradas na auditoria

Estas divergências pertencem ao estado auditado. Não foram corrigidas durante a criação do livro.

INC-01 — Parcela de R\$ 0,00

PENDENTE — severidade alta.

A documentação e `ExpenseService.splitInstallmentAmounts` permitem parcelas zero quando inevitáveis. O teste

`ExpenseFinancialPhase8Test.shouldSplitVerySmallTotalAcrossInstallmentsWithoutLosingCents` confirma:

```
R$ 0,01 em 3x → 0,01; 0,00; 0,00
```

Porém, a V9 contém:

```
CONSTRAINT ck_expense_installments_amount CHECK (amount > 0)
```

Persistir esse exemplo falharia. O teste cobre o algoritmo, não o INSERT.

INC-02 — Motivo do ajuste de parcela

PENDENTE — severidade média.

RN232 e `ExpenseService` exigem `reason`. Na V23, porém, a coluna de ajuste de parcela é nullable e o CHECK aceita `NULL`. Ajuste de fatura, na mesma migration, usa `NOT NULL`.

INC-03 — Cabeçalho legado de receita

DECIDIDO como legado; exposição ainda pendente de harmonização.

Novos RECEIPTS são gravados em `income_movements` e não preenchem `incomes.account_id` ou `received_date`. Algumas respostas ainda leem esses campos do cabeçalho, podendo retornar nulos apesar de existir RECEIPT ativo.

INC-04 — Filtro de `/incomes` por conta

PENDENTE.

`IncomeRepository.searchByUser` filtra a conta do cabeçalho legado. `searchReceivables` usa corretamente `EXISTS` em RECEIPT ativo. Os dois recursos podem responder de modo diferente para receitas criadas após a V30.

INC-05 — Tabela superada ainda presente

Dívida conhecida.

`credit_card_invoice_installments` foi superada por Agreement + settlement + despesa CREDIT_CARD. Tabela, Entity e Repository permanecem; Services não a usam. Sua remoção exigiria migration e decisão sobre histórico.

INC-06 — Lista documental de FKs incompleta

PENDENTE documental.

`docs/23` §265 não enumera várias FKs introduzidas nas fases 9–17, embora elas existam nas migrations e sejam protegidas por testes.

INC-07 — Backfill da V29

PENDENTE para bases legadas.

A V29 obtém `financial_goals.account_id` de contribuições anteriores e depois aplica `NOT NULL`. Uma meta histórica sem contribuição não teria conta para o backfill. Novas metas não sofrem esse problema porque a criação exige conta.

INC-08 — Bloqueios oficiais respeitados

PENDENTE por governança, sem defeito de implementação conhecido.

- `payments.type`: coluna sem valores oficiais, enum ou CHECK;
- edição cadastral de parcela já vinculada a fatura: decisão deferida.

Esses pontos não devem receber implementação provisória.

Glossário

Termos contextualizados ao conteúdo deste volume.

API

Fronteira HTTP REST exposta em /api/v1.

Controller

Camada que traduz HTTP e delega ao Service.

DTO

Contrato de entrada ou saída separado da Entity.

Entity

Classe JPA mapeada para uma tabela.

Flyway

Ferramenta de migrations versionadas.

Hibernate

ORM usado pelo Spring Data JPA.

JWT

Access token HS256 usado na autenticação stateless.

Migration

Alteração versionada e imutável do schema.

Ownership

Isolamento por usuário na aplicação e no banco.

PostgreSQL

Banco relacional oficial do projeto.

Repository

Interface Spring Data de persistência.

Service

Camada de regras e transações.

Testcontainers

PostgreSQL real executado em testes.

UUID v7

Identificador temporal gerado pela aplicação.

Índice remissivo

Referências calculadas pelas ocorrências nas páginas finais; não são estimativas.

API	1, 4, 5, 6, 7	Migration	1, 4, 7, 8, 9, 10, 14, 15, 16
Controller	1, 2, 6, 7, 8	Ownership	2, 3, 4, 7, 8, 12, 13, 14
DTO	1, 2, 3, 4, 7, 8	PostgreSQL	1, 2, 9, 10, 11, 12, 13, 14, 15, 16
Entity	4, 7, 13, 14, 15	Repository	2, 3, 4, 7, 8, 13, 15
Flyway	1, 2, 9, 10, 11, 12, 13, 14, 15, 16	Service	1, 2, 3, 4, 6, 7, 8, 9, 13, 14, 15
Hibernate	1, 2, 9	Testcontainers	1, 7, 9, 14
JWT	1, 2, 4, 6, 7, 8	UUID v7	4, 9